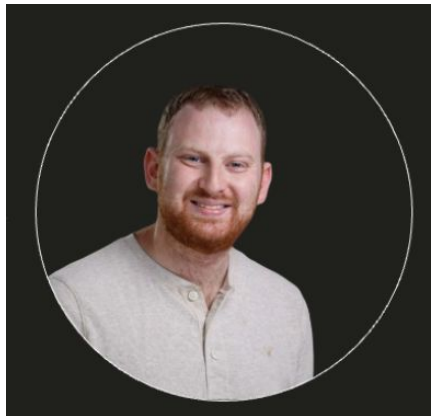




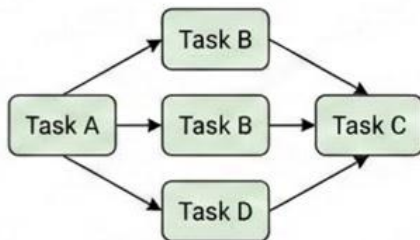
Orchestrating Spark with Airflow

Design tradeoff for cost
and performance



Apache Airflow vs. Apache Spark: Orchestration vs. Execution

Apache Airflow (Orchestration)



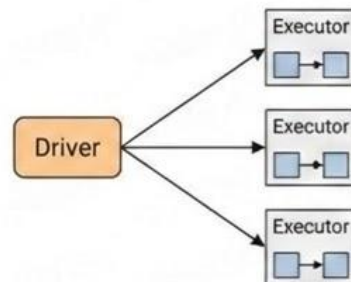
Orchestrates the workflow
DAG; schedules and
monitors tasks.

Spark Declarative Pipeline

Changes things up: Defines what
to do, not how, allowing for
optimization across the DAG.

Bridging Orchestration & Execution

Apache Spark (Execution)



Runs inside each DAG task;
executes distributed data
processing jobs.

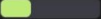
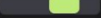
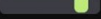
Finding Spark Cost Opportunities in your Airflow steps

- Your Airflow DAG shows all green. Every task completed successfully. And somewhere in your Spark cluster, a job just read 7 billion rows five times in a loop, and nobody noticed.
- Question 1: Which Step Is Eating Your Budget?
- Question 2: Parallel Steps or Sequential? Why Getting It Wrong Is Expensive
- Question 3: Are we using the allocated resources during the life of the step?

Finding the most expensive step

Question 1: Which Step Is Eating Your Budget?

73% of the cost concentrated in 8 steps. All doing essentially the same thing.

Status	Job Name	Timeline	Execution Time	Duration	Input Volume	Cost
Success	ETL_Sales_Pipeline		Today at 08:45	12m44s	1.5 TB	\$48.17
Success	Validate_Customer_Data		Today at 09:00	8m04s	500 GB	\$40.63
Success	Enrich_Product_Catalog		Today at 09:00	8m48s	1.2 TB	\$27.31
Success	Aggregate_User_Sessions		Today at 09:15	12m04s	12 GB	\$27.09
Success	Generate_Analytics_Report		Today at 09:15	12m50s	820 GB	\$22.50
Success	Sync_Delivery_Events		Today at 09:30	5m58s	0 B	\$11.06

Finding the root cause



The root cause was visible the moment we looked at the Spark physical execution plan: each step contained 5 identical Read ORC nodes, each scanning 6.25 billion rows. The same filter chain ran 5 times. Each filtered result was repartitioned separately.

- Detect by repeated Lineage in the physical plan DAG

Parallel Steps vs Sequential

Question 2: Parallel Steps or Sequential: **Why Getting It Wrong Is Expensive:**

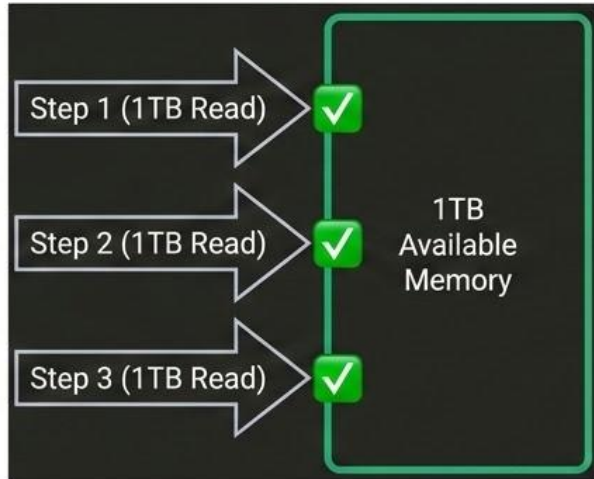
Scenario: job_setup → job_B (52 min) → job_B_report (42 min)

└── critical path: ~94 min

Should we parallelise the 2 critical steps to run 2x faster and cheaper??

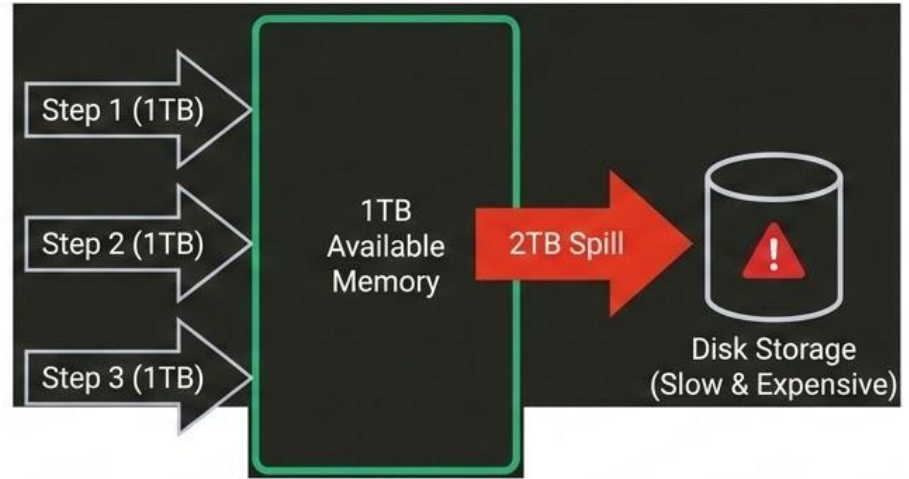
Parallel vs. Sequential: The Memory Dilemma (Scenario 1)

Sequential Execution (Better)



Each step fits entirely into memory.
No disk spill. Fast execution.

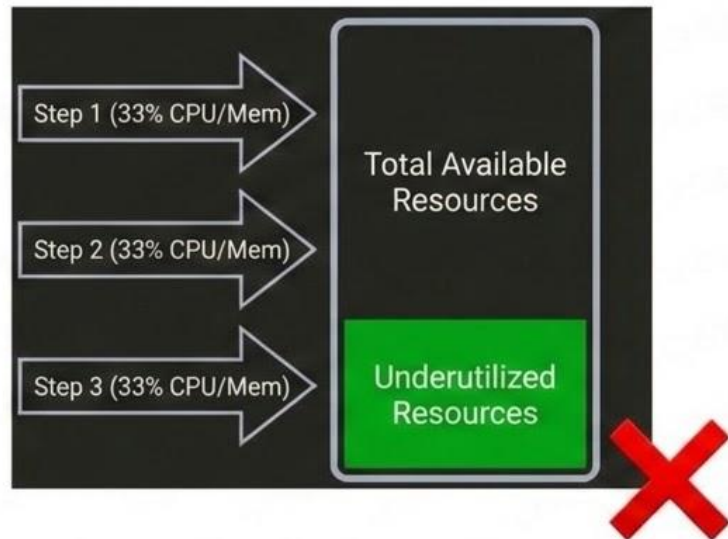
Parallel Execution (Worse)



Total read (3TB) exceeds memory.
Significant spill to disk. Slower & costlier.

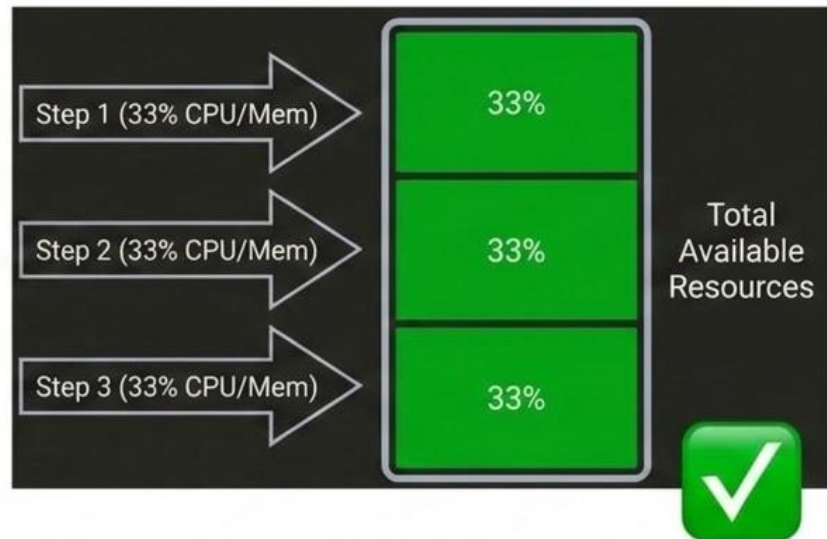
Parallel vs. Sequential: The Resource Dilemma (Scenario 2)

Sequential Execution (Worse)



Slow execution.
Resources wasted.

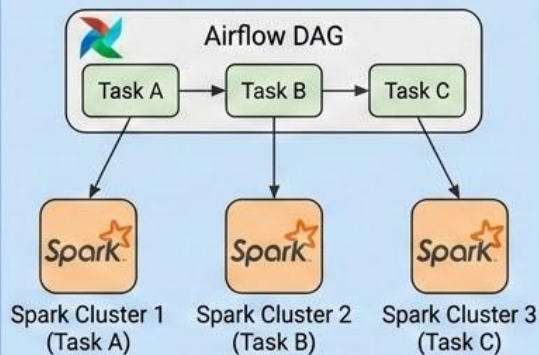
Parallel Execution (Better)



Max Resource Utilization.
Faster execution.

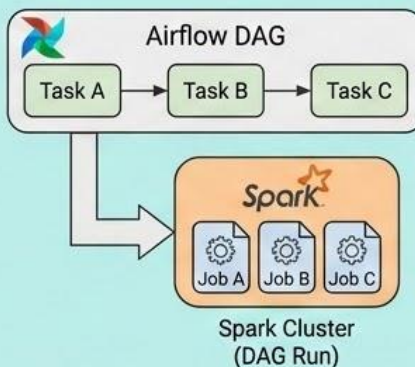
Airflow & Spark Deployment Modes: Cluster Allocation Strategies

1. Dedicated Cluster per Task



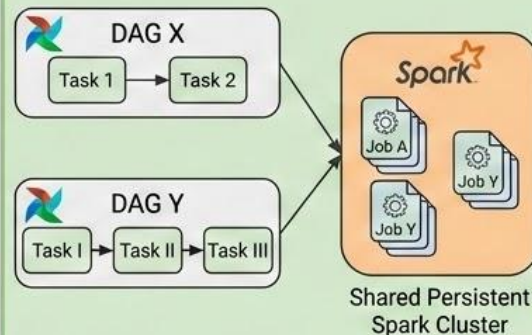
Each individual task execution spins up a new, transient Spark cluster. Maximum isolation, high overhead.

2. Dedicated Cluster per Workflow (DAG)



A single Spark cluster is created for the duration of an entire DAG run. Tasks share the cluster resources.

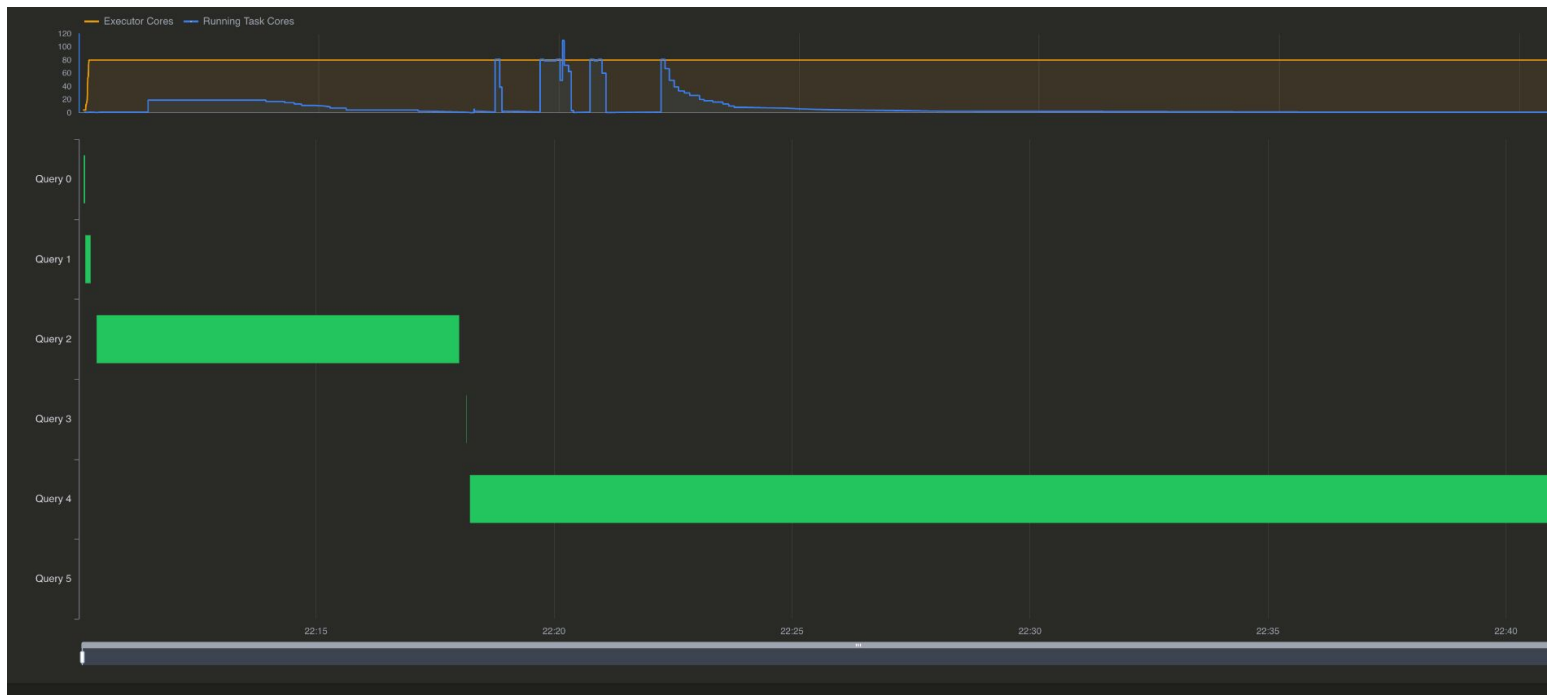
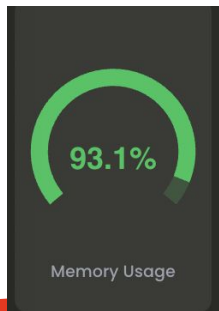
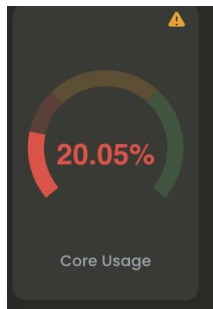
3. Shared Cluster



Multiple DAGs and users submit jobs to a single, long-running Spark cluster. Most efficient, requires multi-tenancy management.

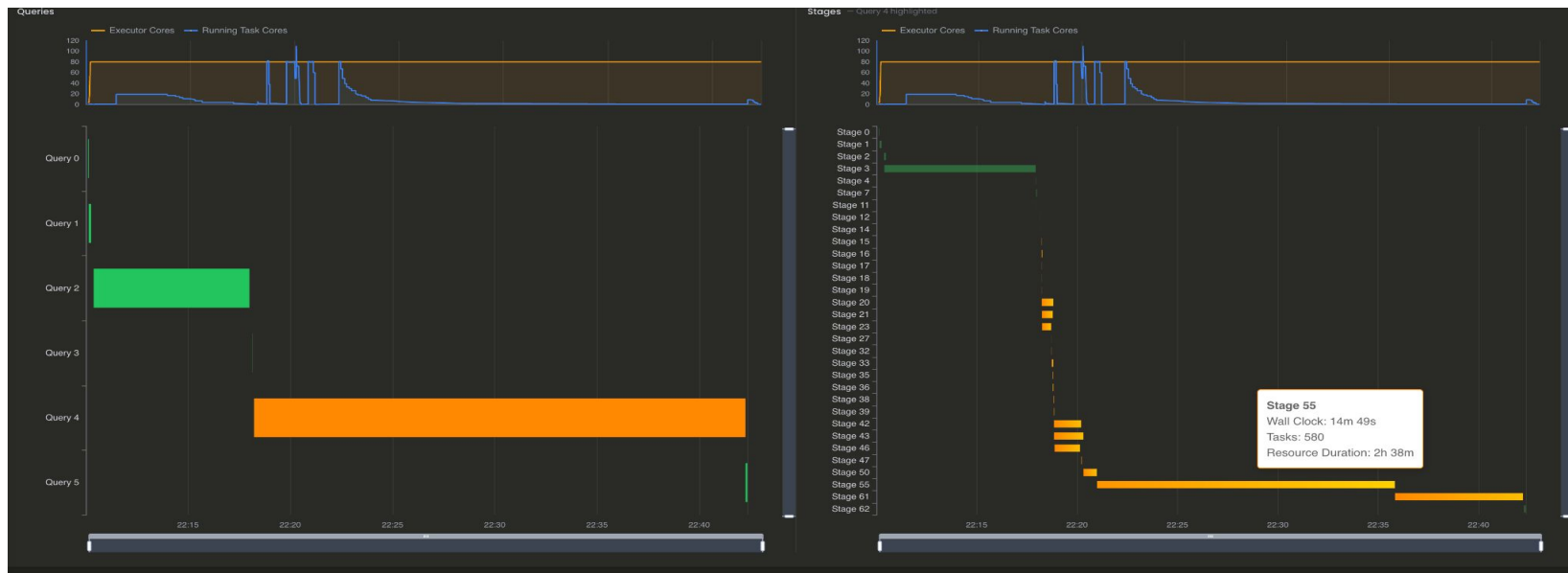
Resources Utilization

Question 3: Are we using the allocated resources during the life of the step?



Inspect the query plan

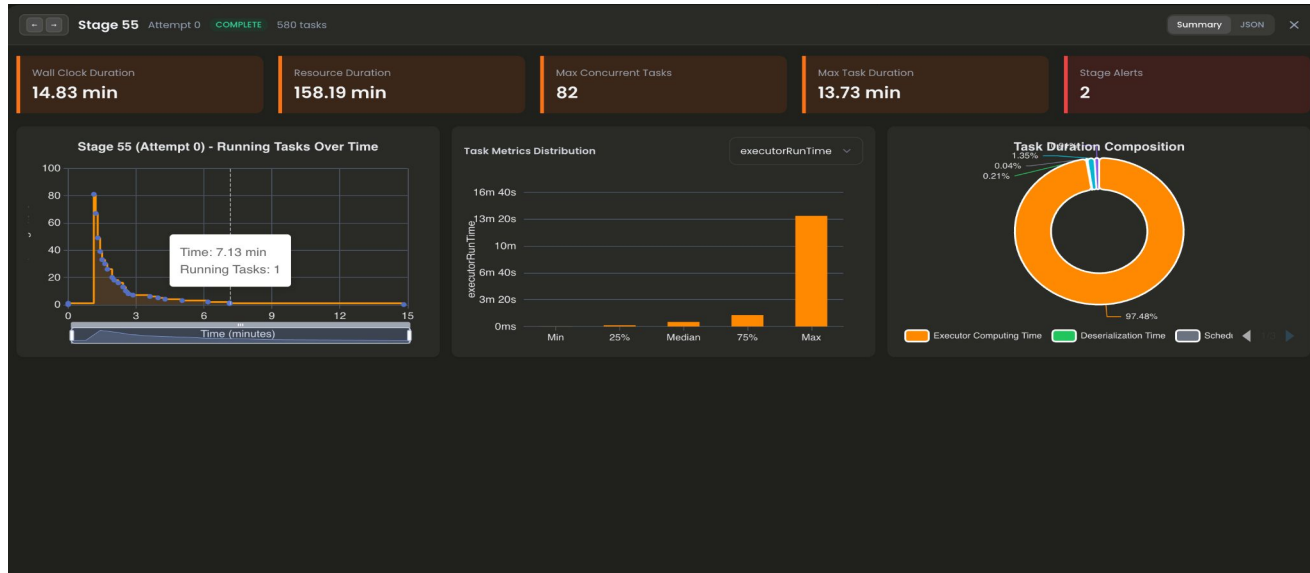
Find the stage with run during idle resources



Inspect the stage

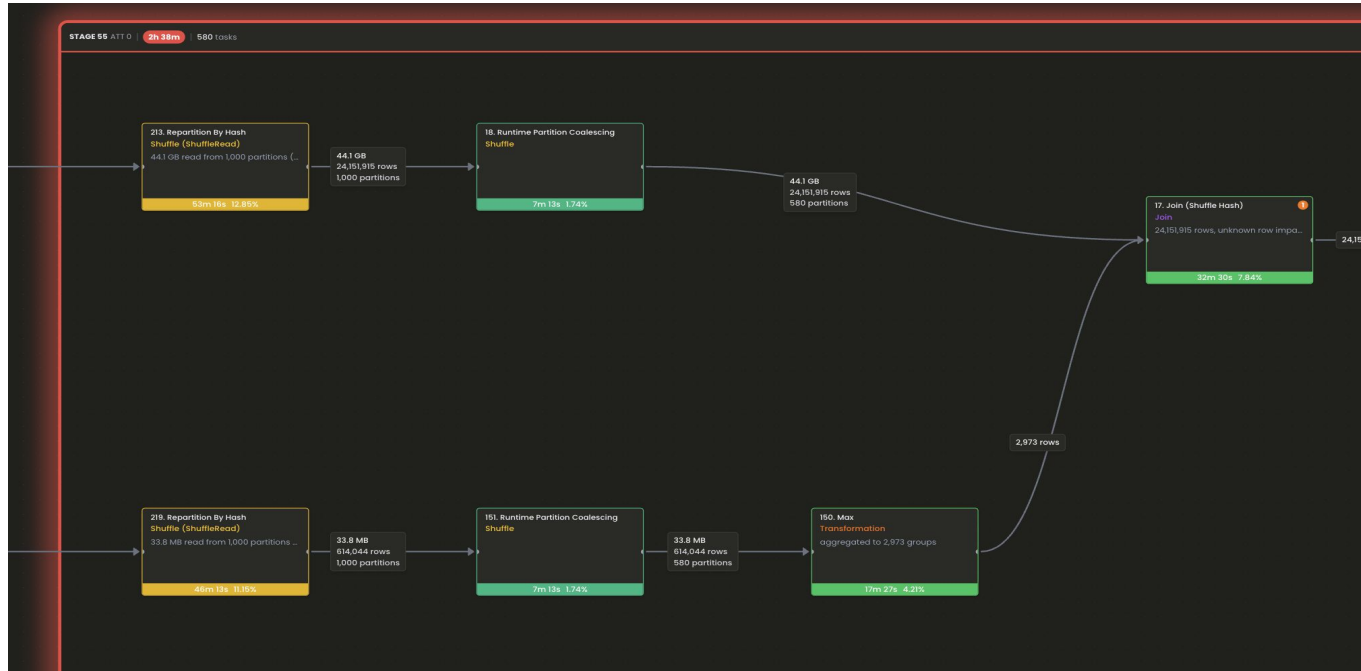
Root cause in this cause is a partition skew on the stage.

But why do we have a partition skew?



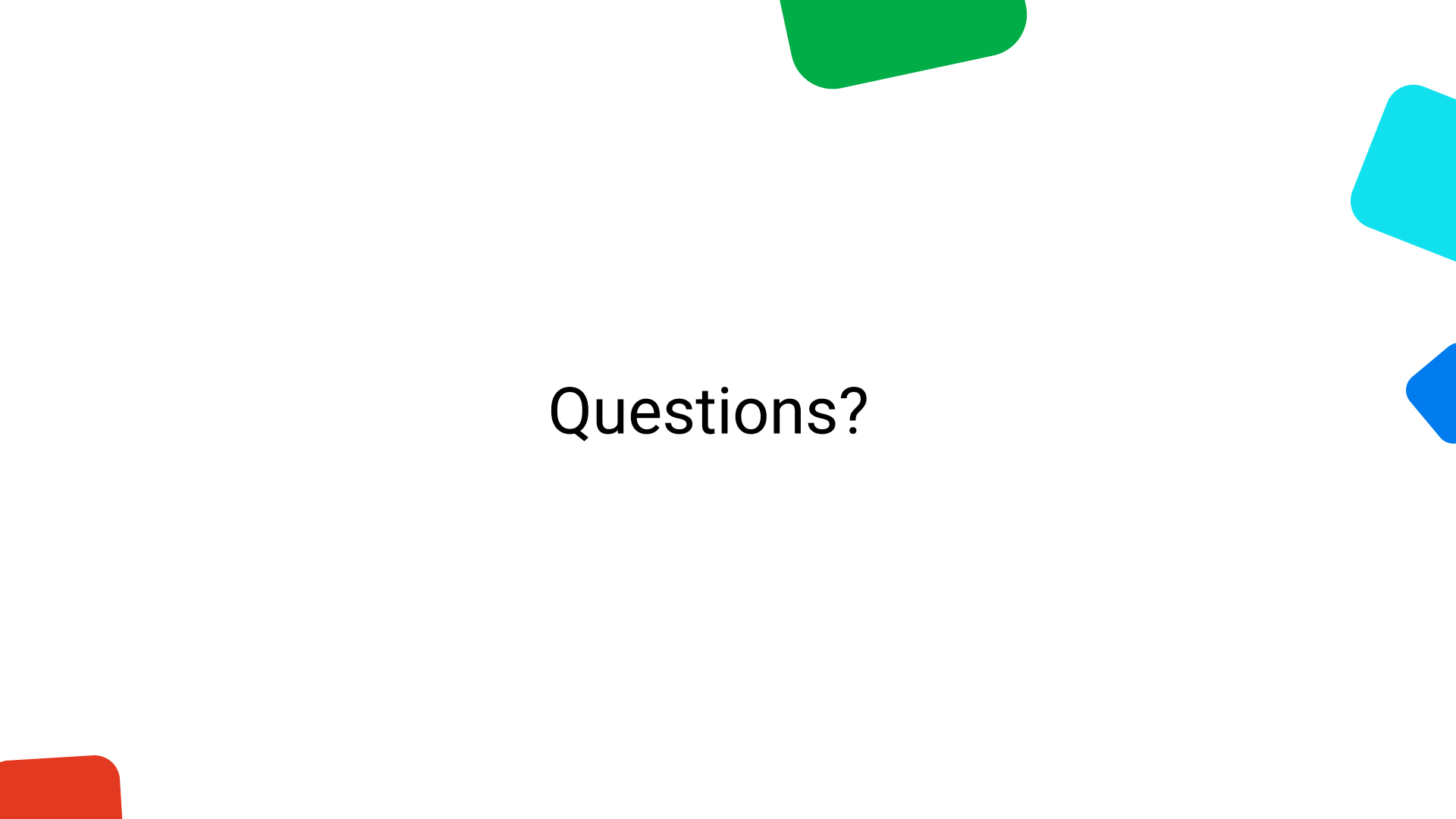
Inspect the stage plan

Shuffle Join with 33Mb and 44.1 Gb



How to bridge the gap

- Propagate your Airflow metadata (DAG IDs, task names, tags) into Spark application tracking, and from there to your compute, observability and finops solutions
- Attach cost to each workflow and step
- Find insights on issues on the spark physical plan and cluster info
- Use AI optimize it for you by giving him as much context as possible
- This is what we do at DataFlint!

The slide features several large, semi-transparent geometric shapes in the corners: a green rounded rectangle in the top center, a cyan rounded rectangle in the top right, a blue rounded rectangle in the middle right, and a red rounded rectangle in the bottom left.

Questions?